

VELoop Rewards

Tap & Earn — Complete MERN Full-Stack Intern Task

Production-oriented implementation specification

Project	VELoop Rewards — Tap & Earn
Stack	MERN: MongoDB + Express.js + React.js + Node.js
Platform body color	#161827
Primary visual direction	Minimal • Premium • Fintech • Interactive • Trusted
Reference	Uploaded Tap & Earn concept + uploaded Tap League screenshots
Primary objective	Build the complete frontend + backend, not a static UI demo

1. Task Objective

Build the complete VELoop Rewards Tap & Earn module as a production-style MERN feature. The experience must feel engaging like a modern tap-to-earn game, but the visual language must remain premium, minimal, trustworthy and fintech-oriented. The implementation must include real backend state, server-authoritative reward calculation, secure tap processing, upgrade systems, seasonal progression, Tap League ranking, reward history and all required UI interactions.

2. Non-Negotiable Requirements

- Use MERN stack throughout the feature.
- Frontend must be React.js and responsive for desktop and mobile.
- Backend must be Node.js + Express.js with MongoDB persistence.
- Do not hard-code balances, energy, rank, reward history, upgrades or season data in React.
- Backend is the source of truth for rewards, balances, energy, cooldowns, upgrades and leaderboard scores.
- Every accepted tap must respect a 200 ms minimum processing interval. Frontend must visibly animate every tap and provide vibration feedback where supported.
- The 200 ms delay must NOT be the only anti-bot mechanism. Backend must reject impossible tap rates and suspicious sequences.
- The UI must never feel cluttered. Advanced features should be understandable at first glance and expandable through drawers/modals.
- Use the supplied Tap & Earn visual concept as the structural inspiration, but do not copy another game's branding or assets.
- Use the platform body color #161827 as the foundation of the design system.

3. Core Tap Interaction

The central interaction is the VE coin/tap circle. Every tap must feel responsive even though reward processing is controlled by the backend.

Requirement	Implementation
Tap animation	Scale/press effect, glow/ripple, small coin/token burst and subtle counter animation.
Haptic feedback	Use navigator.vibrate where supported; fail silently on unsupported devices.
200 ms interval	Do not accept another tap event locally until the 200 ms interaction window is respected.
Server authority	Client sends tap request; server validates state and decides reward.
Reward feedback	Show a compact floating reward such as +1 SVE, +1.2 VE, +25 Tokens or +1 Spin.
No guaranteed reward per tap	The tap animation happens on every accepted tap, but reward outcomes follow configured reward probabilities.
Low-latency feel	Animate immediately; reconcile balance/energy with the server response.
Failed tap	Show subtle feedback such as 'Too fast' or 'Energy empty' without a disruptive modal.

4. Tap Reward Distribution

Use the following initial reward distribution. These values must be configurable from the backend so the economy can be tuned later without changing frontend code.

Outcome	Initial probability	Reward range / rule
SVE	60%	1 SVE per rewarded SVE outcome (initial configuration)
VE	20%	0.6 to 1.7 VE
Spin	2%	1 spin whenever a spin outcome is selected
Gems	5%	0.5, 0.8, 1, 1.2, up to 2 gems
Tokens	13% remainder	5 to 100 tokens

Important: the listed percentages total 87% before the token remainder. Therefore the backend should define Tokens as the remaining 13% so the distribution totals exactly 100%. Do not create an unaccounted reward bucket.

Reward probabilities must be server-side configuration. The client must never decide whether a tap gives SVE, VE, Spin, Gems or Tokens.

5. Energy System

Feature	Requirement
Base energy	500 / 500
Consumption	1 energy per accepted tap before any efficiency rules are applied.
Recharge	+20 energy every 20 minutes by default.
Storage upgrade	User can increase maximum capacity by spending VEs.
Recharge-speed upgrade	User can increase recharge speed using the designated upgrade currency/tokens.
Zero energy	Disable accepted taps until energy becomes available or the user uses an eligible recovery option.
Server calculation	Energy regeneration must be calculated from timestamps rather than requiring a request every 20 minutes.

Cap	Regenerated energy can never exceed maximum storage capacity.
-----	---

6. Energy Storage Capacity

Default capacity is 500. Users can purchase higher capacity. The exact pricing curve should be configurable. Upgrade prices must increase faster than capacity so users cannot create uncontrolled reward inflation.

Suggested initial progression: 500 → 600 → 700 → 800 → 900 → 1000. Keep the values configurable.

7. Multitap

Multitap increases the reward value of an accepted physical tap. Default is x1. Users may upgrade up to x3. The upgrade is temporary and remains valid for 7 days from activation/purchase.

Level	Tap effect	Validity
x1	1x	Default
x2	2x	7 days
x3	3x	7 days

The backend must store activation time and expiry time. After expiry, the user automatically falls back to the highest non-expired multiplier.

8. Recharge Speed

Users can upgrade energy recharge speed. Keep the upgrade curve configurable. Do not allow recharge speed to become unlimited. Suggested initial display: +20/20m → +22/20m → +24/20m → +27/20m → +30/20m.

9. Tap Streak

Show the user's current consecutive tap streak. The streak is primarily an engagement mechanic. Use a visible counter and a lightweight progress effect. Do not make streak rewards large enough to damage the economy.

10. Precision Tap

Occasionally display a small precision target during the tap interaction. A user who taps the target accurately receives Tokens. The reward amount should be configured by the backend. Precision taps must not bypass normal energy or server validation.

11. Mystery Tap

A mystery tap is a special tap outcome that awards SVEs. It should be visually distinct but not excessively flashy. The backend decides when the event is available and the reward amount.

12. Tap Combo

Tap Combo is an animation/engagement layer. It should visually show the user that they are tapping continuously, for example COMBO x12, COMBO x25, COMBO x50. The combo animation itself does not need to create a separate economic reward unless explicitly enabled later.

13. Boost Windows

Provide a temporary 30-second boost window. The boost may increase the tap reward according to backend configuration. It should have a clear countdown, active state and expiry animation. Boost activation must be logged.

14. Lucky Tap + Spin

Lucky Tap cannot trigger before the user reaches a minimum of 300 accepted taps in the relevant tracking window. After 300 taps, the backend may randomly trigger a Lucky Tap. The reward interaction is a Spin.

- Show a clear progress indicator toward the 300-tap threshold.
- Before 300 taps, do not display a misleading 'almost lucky' state that implies a guaranteed reward.
- After eligibility, the backend controls the trigger.
- Spin result must be server-generated and recorded in reward history.
- Prevent double-spinning through idempotent server-side processing.

15. Tap Missions

Create daily/seasonal missions that award Tokens. Examples:

- Complete 100 accepted taps.
- Complete 500 accepted taps.
- Reach a target combo.
- Complete a precision tap.
- Use a boost window.
- Return and tap on multiple sessions.

16. Daily Tap Challenge

Create a daily challenge with convenient, controlled rewards. The user should see progress immediately. The challenge resets according to the platform's daily reset time. Backend must enforce completion and prevent duplicate claims.

17. Energy Bank

Users can purchase an Energy Bank valid for 3 days. The Energy Bank is a separate temporary energy reserve. The exact capacity and price should be configurable. The UI must clearly show its expiry countdown and remaining capacity.

18. Energy Shield

Users can purchase/activate an Energy Shield using SVEs. Shield duration is 10 seconds. During the active window, eligible taps do not consume energy. The shield must have a visible countdown and server-side start/expiry timestamps.

19. Tap Efficiency

Tap Efficiency is a temporary seasonal/monthly upgrade. Initial levels: 1.1x, 1.2x and 1.3x. The user's purchased efficiency expires when the current Tap Season expires. Do not make it permanent.

The frontend must show the current efficiency, current level, upgrade cost and exact expiry date/time.

20. VE Fragments

Introduce VE Fragments as a separate collectible/progression resource. The conversion/value will remain configurable and can be decided later. Build the data model and UI so the value can change without a database migration.

21. Tap Seasons

A Tap Season is a fixed period during which Tap League ranking and seasonal progression are tracked. Season expiry resets temporary Tap Efficiency and starts a new seasonal leaderboard. Keep season configuration in the database.

Season data	Required fields
Identity	seasonId, name, status
Timing	startAt, endAt
Rewards	reward configuration by rank/tier
Economy	tap efficiency rules and seasonal limits
Leaderboard	season-specific accepted tap score

22. Tap League — Reference Analysis

The uploaded reference screenshots show a competitive leaderboard with a clear rank column, player identity area, large central score column, highlighted top ranks, and a persistent 'My Rank' row at the bottom. The important UX patterns to reproduce are:

- Top 100 users are visible in the Tap League.
- Ranks 1, 2 and 3 should have visually distinct premium rank treatments.
- Each row should contain rank, avatar/profile image, display name and Tap Score.
- The current user's row remains pinned/sticky at the bottom even when the user is not in the visible top 100.
- If the current user is inside the top 100, still keep the 'My Rank' presentation consistent and avoid duplicate confusing rows.
- Use a season indicator and time remaining so users understand when rankings reset.
- Leaderboard must be server-generated and paginated/limited to the top 100 plus the current user's rank.
- Do not load an unbounded leaderboard into the browser.

23. Tap League UI Layout

Area	UI requirement
Header	Tap League title, current season name, season countdown.
Top 3	Special rank cards/rows for #1, #2, #3 with premium but restrained visual treatment.
Rows 4–100	Compact leaderboard rows with rank, avatar, username and total accepted taps.
My Rank	Sticky bottom row with highlighted border/background and current user's rank/score.

Empty/loading	Skeleton rows; no layout jump.
Reward preview	Show what top ranks receive at season end.
Mobile	Keep rank and score readable without horizontal scrolling.

24. Tap & Earn Main Screen

The supplied Tap & Earn concept contains a top information area, energy card, tap multiplier, boost timer, large central tap circle, shortcut cards and bottom navigation. Preserve the information hierarchy but redesign it into a polished VELoop fintech interface.

Zone	Must show
Top user card	Avatar/name, level and progress.
Balance	Current VE balance with compact navigation to wallet.
Energy card	Current energy/max energy, recharge rate and next recharge timer.
Multiplier	Current Multitap and Tap Efficiency states.
Boost	Active/inactive 30-second boost and remaining time.
Central tap area	VE coin, tap feedback, combo/streak indicators and subtle background motion.
Feature shortcuts	Daily Challenge, Missions, Lucky/Spin, Energy Bank, upgrades and League where appropriate.
Bottom nav	Home, Tap & Earn, Mine, Wallet, Profile consistent with the existing VELoop platform.

25. Visual Design System

Element	Direction
Base	#161827
Primary accent	Warm VE gold/yellow; use sparingly for value/reward states.
Secondary accent	Electric blue for active/interactive states.
Cards	Deep navy/blue-black surfaces, subtle border, low-opacity glow.
Typography	Modern sans-serif such as Inter/Manrope; clear hierarchy.
Corners	Moderately rounded, not oversized/puffy.
Motion	Short, purposeful transitions; avoid constant particle overload.
Trust	Clear labels, predictable controls, readable values and no misleading reward claims.

The design should be addictive in the sense of being engaging and rewarding, not visually noisy. Avoid excessive neon, giant glows, dozens of floating objects, or casino-like effects.

26. Frontend Technical Requirements

- React.js with reusable components and clear feature modules.
- Use a proper state/data-fetching approach; do not keep authoritative economic values only in local state.
- Responsive layout for desktop and mobile.

- Use CSS modules or the project's existing styling convention consistently.
- Build reusable components for RewardToast, TapButton, EnergyBar, Countdown, BoostCard, UpgradeModal, MissionCard, SpinModal, LeagueRow and StickyMyRank.
- Implement loading, empty, error and retry states.
- Use optimistic animation only; never optimistically add authoritative currency to the balance.
- Synchronize server responses after tap batches or state-changing actions.
- Use accessible buttons, labels, focus states and readable contrast.
- Prevent accidental double submissions on purchase/claim/spin actions.

27. Backend Technical Requirements

- Node.js + Express.js REST API.
- MongoDB with Mongoose.
- Authentication must use the existing VESLoop authentication mechanism; do not create a second unrelated auth system.
- Server-side tap validation and reward selection.
- Server-side energy regeneration from timestamps.
- Server-side upgrade expiry.
- Server-side daily/season reset logic.
- Server-side reward ledger for auditability.
- Atomic balance updates for currencies and reward claims.
- Idempotency for spin, mission claim, daily challenge claim and purchase operations.
- Rate limiting and suspicious-tap detection.
- Centralized configuration for reward probabilities, ranges, cooldowns and season settings.
- Do not trust client-provided reward amounts, energy amounts, tap score or rank.

28. Suggested MongoDB Models

Collection	Important fields
User	profile, level, balances, gems, tokens, spins, currentTapSeasonId
TapState	userId, energy, maxEnergy, rechargeRate, lastEnergyAt, tapMultiplier, multiplierExpiresAt, efficiency, efficiencyExpiresAt, streak, combo
TapEvent	userId, seasonId, timestamp, serverAcceptedAt, rewardType, rewardAmount, energyBefore, energyAfter, requestId
Upgrade	userId, type, level, cost, purchasedAt, expiresAt
Boost	userId, type, startedAt, expiresAt, status
Mission	missionId, seasonId, type, target, reward, activeFrom, activeTo
UserMission	userId, missionId, progress, completed, claimedAt
Spin	userId, seasonId, spinId, eligibleAt, resultType, resultAmount, createdAt
TapSeason	name, startAt, endAt, status, efficiencyRules, rewardRules
TapLeagueScore	seasonId, userId, acceptedTapCount, updatedAt
RewardLedger	userId, source, type, amount, currency, referenceId, createdAt
AdEvent	userId, placement, provider, eventType, createdAt, optionalRewardReference

29. Suggested API Surface

Method	Endpoint	Purpose
GET	/api/tap/state	Return complete current Tap & Earn state.
POST	/api/tap	Validate and process one accepted tap.
GET	/api/tap/history	Return recent reward/tap history.
POST	/api/tap/boost/activate	Activate eligible 30-second boost.
POST	/api/tap/energy-bank/purchase	Purchase 3-day Energy Bank.
POST	/api/tap/shield/purchase	Purchase/activate 10-second Energy Shield.
POST	/api/tap/upgrade	Purchase an eligible upgrade.
GET	/api/tap/missions	Return active missions and progress.
POST	/api/tap/missions/:id/claim	Claim completed mission reward.
GET	/api/tap/daily-challenge	Return today's challenge.
POST	/api/tap/daily-challenge/claim	Claim completed daily challenge.
GET	/api/tap/lucky	Return Lucky Tap eligibility/status.
POST	/api/tap/lucky/spin	Execute one server-authoritative spin.
GET	/api/tap/league	Return top 100 + current user's rank.
GET	/api/tap/season	Return active season and countdown.

30. Tap API Validation Logic

A valid tap should conceptually follow this sequence:

1. Authenticate the user.
2. Generate or validate a unique requestId.
3. Read current TapState using an atomic/transaction-safe approach.
4. Regenerate energy from elapsed time.
5. Check whether the request is too close to the user's last accepted tap.
6. Reject suspicious/impossible request patterns.
7. Check available energy and active shield.
8. Apply efficiency/multitap/boost rules according to current server state.
9. Select reward using server-side probability configuration.
10. Update currency/energy/tap counters and season score atomically.
11. Write RewardLedger and TapEvent.
12. Return the authoritative new state and reward animation payload.

31. Anti-Auto-Clicker / Abuse Protection

- 200 ms minimum accepted-tap interval.
- Server-side timestamp comparison; client timers are not trusted.

- Rate limiting per user/IP/device as appropriate.
- Detect impossible sustained patterns, repeated identical request timing and burst anomalies.
- Use request IDs/idempotency to prevent replaying a tap request.
- Do not accept a client-supplied reward amount.
- Do not accept a client-supplied energy deduction.
- Log suspicious activity for later review.
- Do not block legitimate mobile users simply because their network latency varies; keep detection threshold practical and server-configurable.

32. Ad Integration Boundary

The Tap & Earn system should be ad-ready, but ad rendering must not be allowed to manipulate the economy directly from the client. Ad completion should be verified through the platform's approved ad/verification flow before granting any rewarded benefit.

- Keep ad placements modular so interstitial, banner and rewarded-video providers can be changed later.
- Do not hard-code an ad network SDK throughout the TapButton component.
- Rewarded ad benefits should be explicit: e.g. energy recovery, boost activation or another configured benefit.
- Maintain an AdEvent record for eligible rewarded actions.
- Do not grant an economic reward solely because the frontend says an ad finished.

33. Economy Safety Rules

- Every upgrade is an economic sink; pricing must be configurable.
- Temporary multipliers/efficiency must expire and never become permanent accidentally.
- Season resets must not delete historical ledger data.
- Reward probabilities must be configurable without redeploying React.
- Daily/seasonal caps may be introduced from backend configuration if analytics show excessive issuance.
- All balance-changing operations must be auditable through RewardLedger.
- The system must support changing SVE/VE/Token ranges later without rewriting the tap UI.

34. User Experience Rules

- A new user should understand Energy, Tap, current reward, Multitap, Boost and Season within the first screen.
- Advanced systems such as Energy Bank, Shield, Efficiency and Fragments should be discoverable but not overwhelm the main tap interaction.
- Every purchase modal must show cost, benefit, duration and expiry in plain language.
- Every temporary feature must show an expiry countdown.
- Reward feedback should be immediate and visually lightweight.
- Use consistent terminology: VE, SVE, Tokens, Gems, Spin, Energy, Tap Efficiency, Tap League, Tap Season.

35. Acceptance Criteria

- User can open Tap & Earn and receive complete current state from backend.
- User can tap the central circle and see animation + haptic feedback where supported.
- Accepted taps cannot occur faster than the configured server interval.
- Energy is consumed correctly and regenerated from timestamps.
- All reward outcomes follow backend-configured probabilities and ranges.
- VE, SVE, Tokens, Gems and Spin balances are persisted in MongoDB.

- Multitap, recharge speed, energy capacity and efficiency upgrades work and expire correctly.
- Boost lasts 30 seconds and displays a synchronized countdown.
- Energy Shield lasts 10 seconds and correctly prevents eligible energy consumption.
- Energy Bank lasts exactly 3 days according to server timestamps.
- Lucky Tap cannot be eligible before 300 accepted taps and spin results cannot be duplicated.
- Missions and Daily Tap Challenge track progress server-side and can only be claimed once.
- VE Fragments are stored and displayed as a separate resource.
- Tap Season is persisted with start/end timestamps and controls seasonal expiry.
- Tap League displays top 100 and keeps the current user's rank visible at the bottom.
- Top 3 receive distinct visual treatment.
- Leaderboard ranking is based on accepted server-side taps for the active season.
- All currency-changing actions create auditable ledger entries.
- Refresh/relogin does not reset any progress.
- Frontend cannot directly manipulate authoritative balances.
- Responsive UI works without horizontal overflow on mobile.
- Loading/error/empty states are implemented.
- No feature is left as a fake button or static placeholder.

36. Deliverables From Intern

- Complete React frontend for Tap & Earn.
- Complete Node/Express API.
- MongoDB/Mongoose schemas and indexes.
- Authentication integration with the existing VELoop user system.
- Tap processing and anti-abuse implementation.
- Upgrade, boost, mission, daily challenge, lucky spin, energy bank, shield and season logic.
- Tap League backend and frontend.
- Reward ledger/history.
- Responsive UI matching VELoop's #161827 fintech design direction.
- Environment variable documentation.
- API documentation or Postman collection.
- README with setup, architecture, database setup and test instructions.
- Seed script for development/demo data.
- Unit/integration tests for the most important balance-changing APIs.

37. Recommended Component Structure

Suggested React structure (the intern may improve it if the architecture remains clean):

- TapEarnPage
- TapHeader
- BalanceCard
- EnergyCard
- TapMultiplierCard
- BoostCard
- TapCircle
- TapRewardFloat
- ComboIndicator
- StreakIndicator

- TapShortcuts
- UpgradeDrawer
- EnergyBankModal
- EnergyShieldModal
- MissionPanel
- DailyChallengeCard
- LuckyTapModal
- SpinWheel
- FragmentCard
- TapSeasonCard
- TapLeaguePage
- LeaderboardRow
- MyRankStickyRow
- RewardHistoryDrawer

38. Suggested Backend Structure

- controllers/tapController.js
- controllers/upgradeController.js
- controllers/missionController.js
- controllers/seasonController.js
- controllers/leagueController.js
- services/tapRewardService.js
- services/energyService.js
- services/upgradeService.js
- services/leaderboardService.js
- services/rewardLedgerService.js
- models/TapState.js
- models/TapEvent.js
- models/TapSeason.js
- models/TapLeagueScore.js
- models/RewardLedger.js
- models/UserMission.js
- routes/tapRoutes.js
- config/tapEconomy.js

39. Testing Requirements

- Test energy regeneration after 20 minutes and after long offline periods.
- Test max energy cap.
- Test 200 ms tap validation.
- Test duplicate requestId.
- Test each reward category and range.
- Test probability configuration using a large simulated sample.
- Test multiplier expiry after 7 days.
- Test Efficiency expiry at season end.
- Test 30-second boost expiry.
- Test 10-second shield expiry.
- Test 3-day Energy Bank expiry.

- Test Lucky Tap threshold at 299 and 300 accepted taps.
- Test mission/daily challenge double-claim prevention.
- Test leaderboard ordering and current-user sticky row.
- Test simultaneous requests so balance cannot be double-spent or double-awarded.

40. Final Instruction to Intern

Do not treat this as a UI-only assignment. The expected result is a working full-stack Tap & Earn system. The UI should look polished enough for a product demo, while the backend must be strong enough that a user cannot simply edit JavaScript, send a fake reward amount, replay requests or refresh the page to generate currency. Build the feature in a modular way because VELoop will tune reward probabilities, costs, seasons and limits after launch.

VELoop Rewards • Tap & Earn • MERN Full-Stack Assignment

MORE INFORMATION BELOW!!!

41. Exact Economy Configuration — Final Locked Requirements

This section resolves the remaining implementation ambiguities. These are the initial VELoop Tap & Earn values. Every numeric value must have one authoritative configuration source and must be referenced everywhere else.

41.1 Centralized Configuration Rule

- Create one authoritative Tap & Earn economy/configuration service or database-backed configuration document.
- All prices, probabilities, reward ranges, cooldowns, durations, thresholds, limits, season rules and ad rules must be defined there.
- Frontend must receive display configuration from the backend; it must not hard-code economic values.
- Backend services must read the same configuration source rather than repeating numbers in controllers.
- Admin changes must update the centralized value and all consuming APIs/UI must reflect the change without source-code edits.
- Record every admin configuration change with admin, key, old value, new value, timestamp and reason.
- Version configuration so historical TapEvent and RewardLedger records remain interpretable.

41.2 Tap Reward Roll

Every accepted physical tap performs exactly one server-side reward probability roll.

Outcome	Probability	Initial reward
SVE	60%	1 SVE
VE	20%	Random 0.6–1.7 VE, exactly one digit after decimal
Spin	2%	1 Spin
Gems	5%	Configured values 0.5 / 0.8 / 1.0 / 1.2 / up to 2
Tokens	13%	Random 5–100 Tokens

For ranged decimal rewards, use one decimal place only. Prefer configurable weighted discrete values (for example 0.6, 0.7 ... 1.7) rather than unrestricted floating-point randomness.

41.3 Physical Tap / Multitap Rule

One physical tap is one interaction. Multitap changes the effective tap count and energy consumption: x2 means one physical tap counts as 2 effective taps and consumes 2 Energy; x3 counts as 3 and consumes 3 Energy.

- Store both physicalTapCount and effectiveTapCount.
- Tap League score uses effective accepted taps.
- Energy consumption equals effective tap count unless the active Energy Shield protects part of the tap.
- One physical tap performs one reward probability roll; configured Multitap/Efficiency/Boost modifiers are then applied according to the centralized economy rules.
- TapEvent must record the physical count, effective count, energy consumed and final reward.

41.4 Energy Capacity Pricing

Capacity	Cost
500 → 600	100 VEs
600 → 700	150 VEs
700 → 800	210 VEs
800 → 900	Configurable; must be seeded before production
900 → 1000	Configurable; must be seeded before production

41.5 Multitap Pricing

Tier	Cost	Validity
x1.0	Default	No purchase
x1.1	1000 SVEs	7 days
x1.2	1300 SVEs	7 days
x1.3	Configurable next tier	7 days

Keep the tier list configurable so future tiers can be added without rewriting UI or business logic.

41.6 Recharge Speed Pricing

Recharge	Interval	Cost
+20 Energy	20 minutes	Default
+22 Energy	20 minutes	2000 Tokens
Next tiers	20 minutes	Configurable

41.7 Energy Bank — Final Rule

Property	Requirement
Initial capacity	500 Energy
Purchase #1	100 VEs
Purchase #2	500 VEs
Maximum purchases	2 per active 3-day Bank cycle
Recharge	+20 Energy every 120 minutes
Priority	Bank Energy is consumed before normal Energy
Validity	3 days
Expiry	Remaining Bank Energy is lost
Stacking	No duration stacking; second purchase adds another 500 Bank Energy capacity, up to 1000 total during the same active cycle

Here, stacking means combining active purchases. V1 allows two purchases but does not extend the 3-day expiry. The second purchase can increase active Bank capacity from 500 to 1000.

41.8 Energy Shield — Final Rule

Property	Requirement
Cost	100 VEs
Duration	30 seconds
Protection	90% of eligible taps consume no Energy; 10% consume normal Energy
Minimum Energy required	None
Active shields	1 maximum
Purchase while active	Blocked
Cooldown after use	5 minutes
Stacking	Not allowed

The server decides the protected/charged tap outcome; the client must never decide which 10% consume Energy.

41.9 Tap Efficiency — Final Rule

Efficiency	Cost	Validity
x1.0	Default	Current season
x1.1	1.1 SVEs	Current Tap Season
x1.2	1.2 SVEs	Current Tap Season

x1.3	Configurable	Current Tap Season
------	--------------	--------------------

Tap Efficiency expires at season end. Implement its effect as a configurable multiplier on eligible reward amounts, with currency-specific rounding. Do not multiply Spins or unrelated outcomes unless explicitly enabled in configuration.

41.10 Tap Streak vs Tap Combo

System	Purpose	Reset
Tap Streak	Longer-term engagement/progression	5 seconds inactivity
Tap Combo	Short rapid-tapping visual effect	2 seconds inactivity

They must use separate state variables and separate reset timers.

41.11 Mystery Tap

After every 250 effective taps, the user receives one Mystery Tap eligibility roll. There is a 0.5% chance of triggering the Mystery Tap at that milestone. A successful Mystery Tap awards an SVE amount configured by the backend. The milestone and attempt are server-tracked.

41.12 Lucky Tap / Spin Reward Table

Outcome	Probability
50 Tokens	25%
500 VEs	0% — disabled, configurable for future
20 VEs	5%
2 Spins	10%
10 Gems	8%
100 SVEs	12%
50 SVEs	25%
Better Luck	10%
₹5 Amazon Voucher	0% — disabled, configurable for future
+15 Energy	5%

These probabilities total exactly 100%. The server creates and records the spin result before the client reveals the animation result.

41.13 Tap League Final Rewards

Rank	Season reward
#1	10,000 VEs + 5 Spins
#2	5,000 VEs + 3 Spins
#3	1,500 VEs + 1 Spin
#4–10	500 VEs + 2,500 Tokens
#11–25	5,000 SVEs + 1,000 Tokens
#26–50	2,500 SVEs + 10 Gems
#51–100	1,000 SVEs + 500 Tokens

Tie-breaker: higher effective accepted tap score ranks first; if scores are equal, the user who reached that score earlier ranks first; if still equal, use a deterministic database identifier.

Season rollover: freeze scoring → calculate ranks → distribute rewards exactly once → archive the season → activate the next season → reset seasonal efficiency. Historical leaderboard and reward ledger data must remain available.

42. Ad System — Required Demo Implementation

Build a modular demo ad system. The admin controls the tap window in which an ad opportunity can appear, with the initial range set to 15–40 accepted/effective taps.

- Central configuration keys: minAdTapThreshold = 15 and maxAdTapThreshold = 40.
- Use randomized opportunity timing inside the configured range and apply sensible frequency caps.
- Create a replaceable DemoAdProvider abstraction.
- Support demo Interstitial, Full-page/Video and Banner placements.
- Demo ads must be clearly marked as demo/non-production.
- Do not put ad-network SDK logic inside TapCircle/TapButton.
- Track opportunity, shown, completed, skipped, failed and rewarded events.
- Rewarded-ad completion must be verified by a trusted callback/server mechanism before granting an economic benefit.
- Changing the centralized ad configuration must update every consuming UI/API.

43. Admin Control Center — Admin-Proof Architecture

The existing VELoop Admin Dashboard must be able to monitor and configure the entire Tap & Earn system. Build a dedicated Tap & Earn admin module or expose a clean admin API for the existing dashboard.

Admin area	Monitor/configure
Economy Config	Probabilities, reward values/ranges, prices, multipliers, durations, thresholds and limits
Tap Analytics	Physical taps, effective taps, accepted/rejected taps, active tappers, taps per hour/day
Rewards	SVE/VE/Token/Gem/Spin issuance and distribution
Energy	Consumption, recharge, capacity upgrades, Bank and Shield usage
Upgrades	Purchases, active/expired upgrades and sink totals
Ads	Opportunities, impressions, completion/failure and placement performance
Anti-Bot	Rejected taps, rate violations, suspicious patterns and risk flags
Tap League	Current season, top 100, score distribution and reward status
Seasons	Start/end, active status, rollover/finalization
Reward Ledger	Searchable user/system balance changes
Config Audit	Admin, changed key, old value, new value, timestamp, reason

No economic value may be duplicated across files. Admin edits must validate, persist, audit and propagate through the same configuration source.

44. UI/UX — HIGHEST PRIORITY

The UI is the highest-priority part of this assignment because the user experiences Tap & Earn almost entirely through the interface. The intern must treat visual quality, interaction feedback, responsiveness and clarity as functional requirements.

- First viewport must clearly communicate VE balance, Energy, recharge countdown, Multitap, Tap Efficiency, Boost and the central Tap Circle.

- Central Tap Circle is the hero interaction and must feel physically responsive on every accepted tap.
- Every tap: press/compression animation + ripple/glow + reward float + subtle haptic feedback where supported.
- Use the supplied VE tap-circle concept as interaction inspiration while using VELoop's own #161827 fintech design language.
- Use VE gold as the primary reward accent and blue as a secondary interaction accent.
- Do not make every card glow. Reserve stronger effects for active rewards, boosts, Lucky/Mystery events and the Tap Circle.
- Use bottom sheets, drawers and focused modals for advanced systems so the first screen stays understandable.
- Every purchase must show cost, benefit, duration and expiry before confirmation.
- Every temporary feature must have a live countdown.
- Use consistent visual identity for SVE, VE, Tokens, Gems, Spins and Energy.
- Show a compact last-reward/history feedback area without forcing the user away from the tap screen.
- Loading, error, rejected tap, reconnecting and empty states must be designed—not plain text placeholders.
- Never freeze the Tap Circle while waiting for a network response; animate immediately and reconcile with authoritative server state.
- Rejected taps should show subtle reasons such as Too Fast, Not Enough Energy or Cooldown Active.
- Touch targets must be mobile-friendly and the Tap Circle must not conflict with bottom navigation.
- Respect reduced-motion preferences where practical.
- Avoid casino-like effects, excessive particles, flashing, giant gradients or visual clutter. Target: engaging fintech, not cheap game UI.

44.1 First-View Information Hierarchy

Priority	Information
1	Tap Circle + current reward feedback
2	Energy current/max + recharge countdown
3	VE balance + level
4	Multitap + Tap Efficiency
5	Boost + countdown
6	Daily Challenge + Missions
7	Lucky/Spin + Energy Bank + Shield
8	VE Fragments + Tap Season
9	Tap League

44.2 Tap Feedback Sequence

1. User touches Tap Circle.
2. Frontend immediately plays press animation and vibration where supported.
3. Frontend begins the 200 ms interaction lock.
4. Unique requestId is sent to backend.
5. Backend validates authentication, timing, energy, Shield, anti-bot and season state.
6. Backend performs one reward roll and applies configured Multitap/Efficiency/Boost rules.
7. Backend atomically updates state and writes TapEvent + RewardLedger.
8. Frontend receives authoritative response.
9. Reward float displays exactly what was awarded.
10. Balance, Energy, Combo, Streak and League progress animate to the returned state.

45. Single-Source Configuration Architecture

All values below are examples of configuration keys, not duplicated constants. The intern must implement a single source of truth.

- tapEconomy.reward.sve
- tapEconomy.reward.ve
- tapEconomy.reward.tokens
- tapEconomy.reward.gems
- tapEconomy.reward.spin
- tapEconomy.energy.baseCapacity
- tapEconomy.energy.recharge
- tapEconomy.energy.capacityTiers
- tapEconomy.multitap.tiers
- tapEconomy.rechargeSpeed.tiers
- tapEconomy.energyBank
- tapEconomy.energyShield
- tapEconomy.tapEfficiency.tiers
- tapEconomy.tapStreak
- tapEconomy.tapCombo
- tapEconomy.mysteryTap
- tapEconomy.luckyTap.spinRewards
- tapEconomy.ads
- tapEconomy.seasons
- tapEconomy.league.rewards
- tapEconomy.security

46. Reliability, Precision and Concurrency

- Use integer smallest currency units or MongoDB Decimal128 for fractional balances; never rely on binary floating-point arithmetic for authoritative balances.
- Use unique indexes for userId/requestId where replay protection requires them.
- Index TapEvent by userId + timestamp and seasonId + timestamp/score as appropriate.
- Tap processing must be atomic so simultaneous requests cannot double-spend Energy or double-award rewards.
- Configuration changes must not alter historical ledger events.
- Store timestamps in UTC; convert to local time only in the UI.
- Server time is authoritative for all cooldowns, recharge, expiry and seasons.
- State-changing endpoints should return authoritative resulting state or a version number for safe frontend reconciliation.

47. Final UI Acceptance Checklist

- Premium, attractive first load; no generic template appearance.
- Tap Circle is dominant but does not hide important information.
- Tap feedback is visible immediately.
- Reward types are understandable at a glance.
- Energy and recharge state are instantly understandable.
- Multitap, Efficiency and Boost are visually distinct.
- Temporary features communicate expiry.
- Ads do not cause layout jumps or destroy the core interaction.

- Tap League feels competitive and polished with Top 3 and sticky My Rank.
- Mobile has no horizontal scrolling and controls are thumb-friendly.
- Desktop uses space naturally without oversized stretched cards.
- Loading, network failure, rejected tap and success states are designed.
- Animations are smooth, short and purposeful.
- The interface remains readable when multiple effects are active.
- Every visible economy value comes from backend state/configuration.

48. Final Definition of Done

The feature is complete only when an authenticated user can tap, consume Energy, receive server-selected rewards, purchase and expire upgrades, use Boost/Bank/Shield/Efficiency, complete Missions and Daily Challenges, trigger Mystery/Lucky mechanics, accumulate seasonal Tap League score, see Top 100 + My Rank, encounter the configurable demo ad system, and have every economic event visible/auditable from the Admin Dashboard. The UI must be polished, responsive, engaging and production-facing—not an intern prototype.

Project Timeline s Submission Deadline

To ensure smooth project execution and timely review:

Project Assigned On:	20 August 2026
Project Start Date:	20 August 2026
Submission Deadline:	10 September 2026
Submission Time:	5:30 P.M. (IST)